1 Wprowadzenie

> Wybudować grób. To się tylko tak mówi. A kto nie budował, ten nie wie, co taki grób kosztuje. Prawie tyle co dom. Choć grób, mówią, też dom, tylko że na tamto życie. Bo wieczność nie wieczność,
> a swój kąt powinien człowiek mieć
>
> Wiesław Myśliwski, *Kamień na kamieniu*

„(...) Bo wieczność nie wieczność, a swój kąt powinien człowiek mieć. (...)" napisał Wiesław Myśliwski w Kamieniu na kamieniu. I choć to zdanie pojawia się tutaj tylko dlatego, że czasami trzeba efektownie zacząć, to... może to wcale nie jest przypadek?

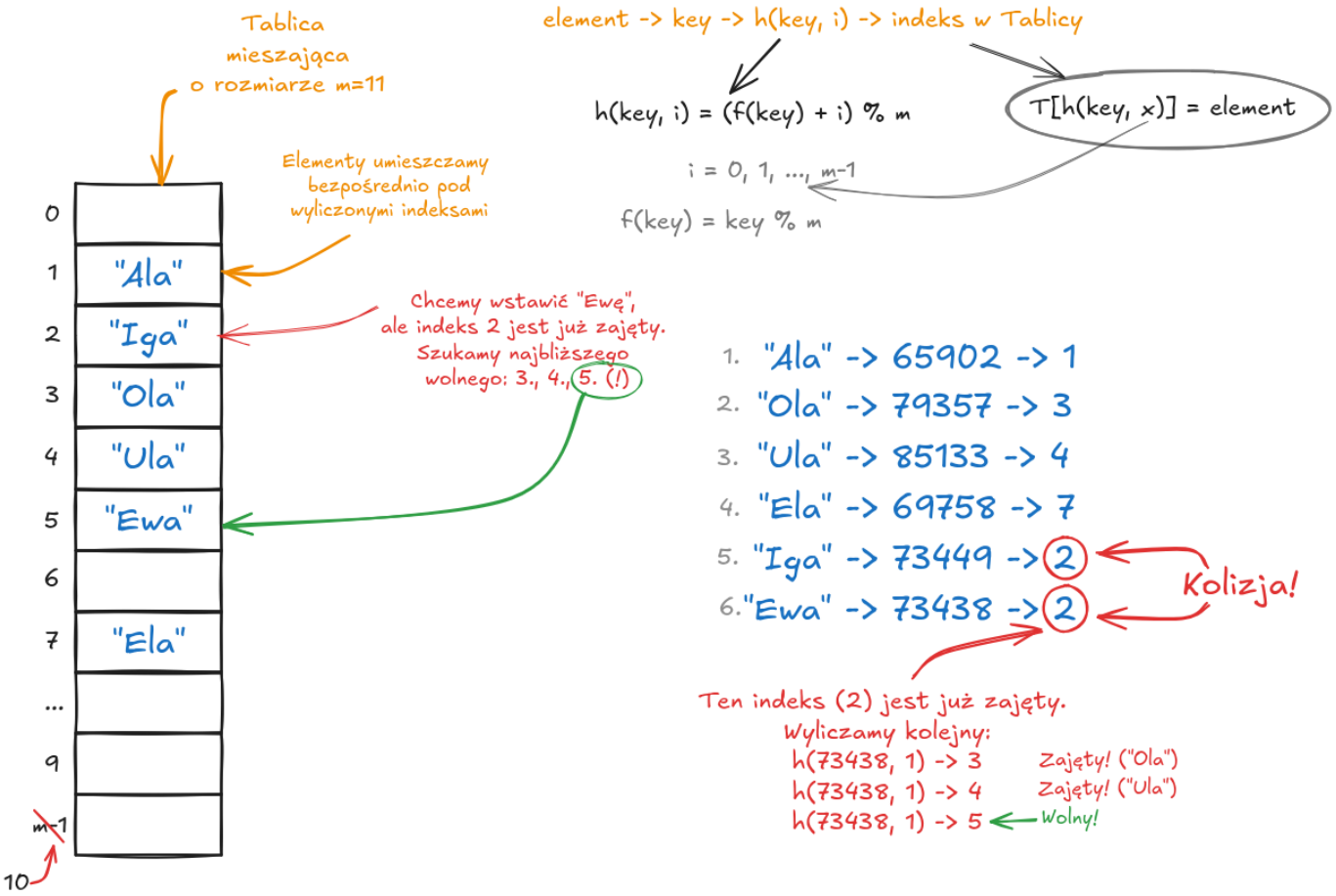
Tablice mieszające z rozwiązywaniem kolizji metodą łańcuchową w teorii powinny zapewnić nam wykonywanie operacji (dodawania, wyciągania i usuwania elementów) ze stałą złożonością $O(1)$ niezależną od liczby przechowywanych elementów. Tyle teoria. W praktyce sprawdziliśmy, że tablice są efektywne, a przy odpowiedniej implementacji funkcji haszujących udaje się spełnić warunek oczekiwanej złożoności czasowej, na którą oczywiście wpływa aktualna wartość współczynnika zapełnienia tablicy α, czyli stosunek liczby wszystkich wstawionych elementów do rozmiaru tablicy mieszającej. Wadą rozważanej przez nas implementacji było występowanie kolizji początkowo zgodne z prawdopodobieństwem jego wystąpienia (gdy $\alpha < 1$), a w późniejszej fazie wymuszone przez brak wolnych indeksów (czyli pustych łańcuchów) w naszej tablicy (gdy $\alpha > 1$).

A co mogłoby się stać, gdyby każdy element – niczym człowiek u W. Myśliwskiego – „swój kąt powinien mieć"? Postawmy to pytanie nieco bardziej przyziemnie: czy przy rozwiązywaniu kolizji trzeba korzystać z metody łańcuchowej? Nie czekając zbyt długo możemy odpowiedzieć: nie, nie trzeba. Innym pomysłem na poradzenie sobie z sytuacją, gdy dochodzi do kolizji (gdy dla dwóch kluczy (elementów) otrzymujemy taką samą wartość haszującą) jest **adresowanie otwarte**.

W tym podejściu wszystkie elementy są wprost przechowywane w tablicy, czyli nie tworzymy żadnych list do przeszukiwania i łączenia kluczy (elementów) o takiej samej wartości haszującej. W takim razie, jeśli elementy są umieszczane bezpośrednio w tablicy pod wyliczonymi indeksami, to należy sprawdzić, co się stanie, gdy wystąpi kolizji (która niewątpliwie nie została cudownie wyelimionwana przez samą zmianę nazwy metody).

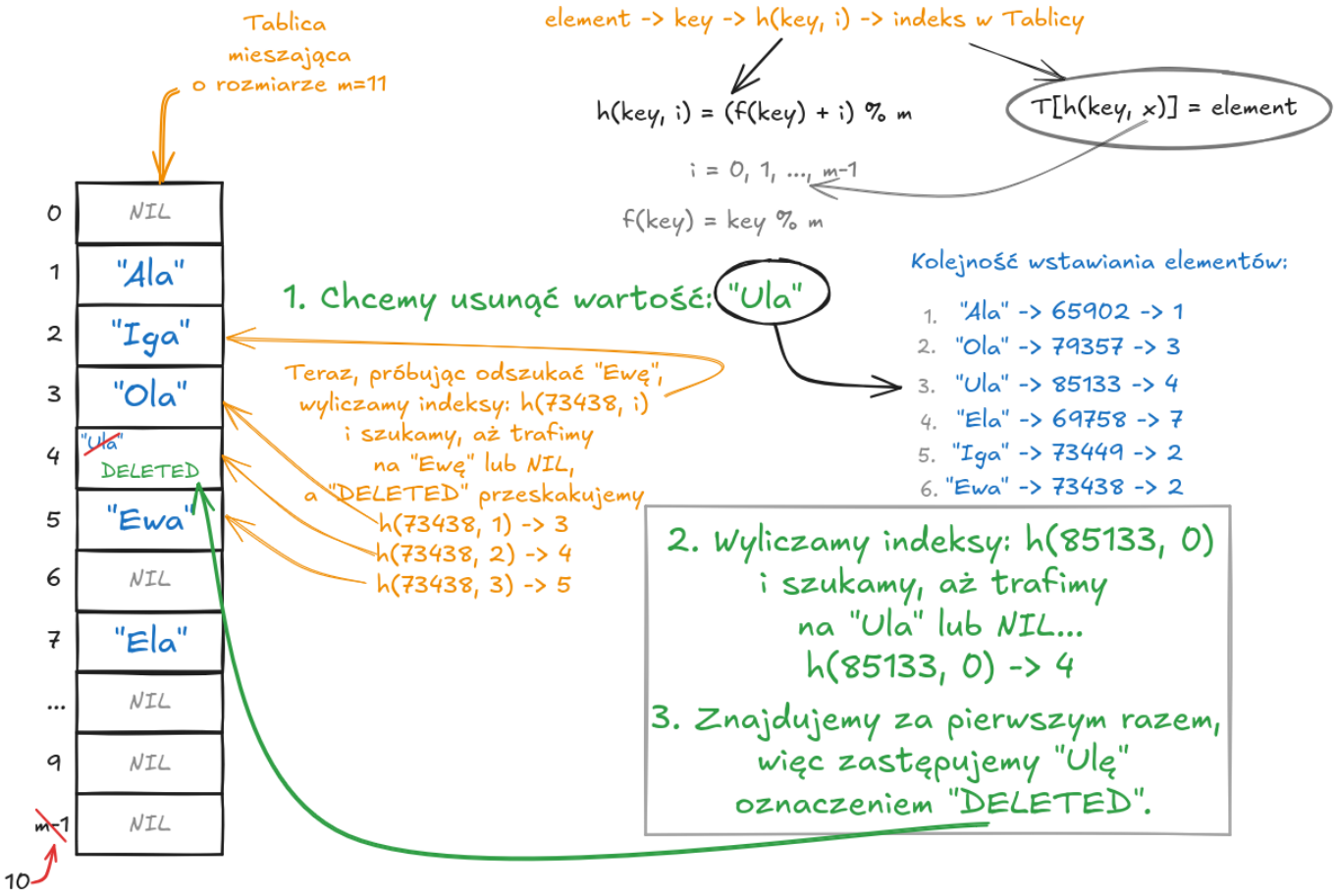
Adresowanie otwarte, zamiast rozważania pojedynczego indeksu wyliczonego na podstawie klucza, oblicza ciąg pozycji (na podstawie klucza), które są potencjalnymi indeksami (miejscami) docelowymi dla analizowanego elementu (klucza). A z tego ciągu potencjalnych indeksów, w przypadku wstawiania nowego elementu do tablicy mieszającej, wybierany jest pierwszy wolny (nie-

zapisany) indeks. Przy wyszukiwaniu proces wygląda podobnie: przeszukujemy ciąg indeksów (wyznaczonych na podstawie klucza), pod którymi potencjalnie może być umieszczony poszukiwany element. Jeżeli po drodze trafimy na puste miejsce w tablicy lub przejrzymy wszystkie indeksy (w wyznaczonej kolejności), to stwierdzamy, że poszukiwany element nie występuje w tablicy. Zostało to pokazane na rysunku 1.



Rysunek 1: Przedstawienie procesu wstawiania kolejnych elementów do tablicy mieszającej. Pierwsze wartości (Ala, Ola, Ula, Ela, Iga) trafiają bezpośrednio pod wyliczone indeksy. W przypadku ostatniej wstawianej wartości dochodzi do kolizji (indeks 2. wyliczony dla „Ewy" jest już zajęty), dlatego potrzebne jest przejęcie przez kolejne wartości ciągu kontrolnego. Ostatecznie „Ewa" zostaje umieszczona pod indeksem 5. (po sprawdzeniu: 2., 3., 4.,). Do wyliczenia indeksów została użyta funkcja adresowania liniowego przy rozmiarze tablicy $m = 11$.

Sprawa nieco bardziej komplikuje się, gdy usuwamy element z tablicy mieszającej. Wtedy nie wystarczy odnaleźć indeksu, pod którym znajduje się poszukiwany element, a następnie zamienić go na wartość `NIL` (oznaczającą wartość pustą). Dlaczego? Ponieważ usunięty element może zaburzyć porządek poszukiwania innych elementów, które miały z nim kolizję (i zostały umieszczone na dalszych pozycjach ze swoich wyliczonych ciągów). Dla lepszego zrozumienia tej sytuacji, spójrzmy na rysunek 2.



Rysunek 2: Przedstawiono dwie operacje: usunięcia wartości „Ula”, odszukania wartości „Ewa”. „Ulę” odnajdujemy od razu i usuwamy ją poprzez wpisanie do tablicy pod indeksem $i = 4$ wartości „DELETED”. Dzięki temu podczas wyszukiwania wartości ta komórka zostanie potraktowana jako zajęta, a podczas wstawiania nowego elementu – jako pusta. Szukając „Ewy” przeskakujemy komórkę pod indeksem $i = 4$ i odnajdujemy „Ewę” pod indeksem $i = 5$.

W związku z tym usuwany element należy zastąpić innym, specjalnym ele-

mentem, który będzie oznaczał, że w przypadku operacji wyszukiwania (innego elementu) nie należy przerywać procesu (uznajemy, że ten indeks nie jest pusty, a po prostu nie odpowiada wyszukiwanemu elementowi), a w przypadku operacji wstawiania ten element będzie traktowany jako oznaczenie pustego (wolnego) miejsca w tablicy mieszającej.

1.1 Obliczanie ciągów kontrolnych

W adresowaniu otwartym zwykle stosuje się jeden z trzech popularnych sposobów wyliczania ciągów kontrolnych, a zalicza się do nich: adresowanie liniowe, adresowanie kwadratowe, dwukrotne haszowanie.

Przy założeniu, że rozmiar tablicy mieszającej jest równy m, wyliczane ciągi kontrolne będą permutacjami wszystkich indeksów tablicy mieszającej: $\{0, 1, ..., m-1\}$.

1.1.1 Adresowanie liniowe

Wykorzystując funkcję f_1 haszowania modularnego (charakterystyczną dla meteody łańcuchowej), można przyjąć, że ciąg indeksów w adresowaniu liniowym jest wyliczany jako:

$$h(k,i) = (f_1(k) + i)\%m$$
$$f_1(k) = k\%m$$

, dla $i = 0, 1, ..., m-1$.

Niewątpliwą zaletą tej metody jest łatwość implementacji. Wadę zaś stanowi występowanie zjawiska **grupowania pierwotnego**, które wynika z faktu, że im większa jest wartość współczynnika zapełnienia (im więcej elementów w tablicy), tym dłuższe ciągi zajętych indeksów występują w tablicy mieszającej, a tym samym element wstawiany jest na pierwszą wolną pozycję po przeszukaniu wielu wcześniejszych, zajętych indeksów. To wprost przyczynia się do wydłużania średniego czasu trwania każdej operacji (wstawiania, wyszukiwania, usuwania).

1.1.2 Adresowanie kwadratowe

Z problemem grupowania lepiej radzi sobie adresowanie kwadratowe, w którym skok pomiędzy kolejnymi indeksami (wartościami wyliczonego ciągu kontrolnego dla danego klucza k) jest zależny od kwadratu numeru indeksu rozpatrywanej wartości ciągu kontrolnego:

$$h(k,i) = (f_1(k) + a \cdot i + b \cdot i^2)\%m$$
$$f_1(k) = k\%m$$

, dla $i = 0, 1, ..., m-1$. Gdzie dodatkowy problem stanowi dobór stałych: a, b.

Chociaż kolejne wartości są od siebie bardziej oddalone niż w przypadku adresowania liniowego, to w tym podejściu liczba różnych możliwych ciągów kontrolnych również jest równa m (jest „tylko” m indeksów, od których możemy

rozpocząć wyliczenie ciągów kontrolnych). Dodatkowo jeśli dla dwóch różnych kluczy wystąpi kolizja ($h(k_1, 0) == h(k_2, 0)$), to pozostałe wartości ciągu kontrolnego, również będą ze sobą kolidowały ($h(k_1, i) == h(k_2, i)$), co w efekcie doprowadzi do (niepożądanego) **grupowania wtórnego**.

1.1.3 Dwukrotne haszowanie

Teoretycznie remedium na problemy („małą" liczbę ciągów kontrolnych, grupowanie elementów) jest funkcja dwukrotnego haszowania, w której – jak nazwa wskazuje – wykonuje się wyliczanie hasza dwa razy (przy pomocy dwóch różnych funkcji):

$$h(k, i) = (f_1(k) + i \cdot f_2(k))\%m$$
$$f_1(k) = k\%m$$
$$f_2(k) = 1 + (k\%(m-1))$$

, dla $i = 0, 1, ..., m-1$.

Podane funkcje: f_1, f_2 stanowią oczywiście jedynie przykład. Istotne jest jednak zwrócenie uwagi na kilka szczegółów. Aby upewnić się, że otrzymany ciąg kontrolny będzie permutacją wszystkich indeksów tablicy mieszającej, należy zadbać o to, aby wartość f_2 była względnie pierwszą z m ($NWD(f_2(k), m) = 1$), a można to osiągnąć czyniąc jak w powyższym przykładzie (przyjąć m jako liczbę pierwszą, a wartość f_2 ograniczyć do wartości mniejszych od m ($m-1$). Przy spełnieniu tych warunków funkcja dwukrotnego haszowania powinna dostarczać $\Theta(m^2)$ ciągów kontrolnych – każda para $(f_1(k), f_2(k))$ daje inny ciąg kontrolny.

2 Zadania

2.1 Zadanie 0

Proszę uporządkować swoje repozytorium, aby organizacja projektów wyglądała jak poniżej. Oczywiście w tej strukturze mogą znajdować się również inne katalogi i pliki, ale proszę dokładnie odwzorować przedstawiony porządek projektów.

```
.korzeń_repozytorium
+-- CW_0
|   |
|   +-- pliki_i_katalogi_z_zajęć
|
+-- CW_1
|   +-- AiSD2026ZEx1
|       +-- pom.xml
|
+-- CW_2
```

```
|   +-- AiSD2026ZEx2
|       +-- pom.xml
|
+-- CW_3
    +-- AiSD2026ZEx3
        +-- pom.xml
```

2.2 Zadanie 1

Proszę porównać czas działania i liczbę wykonanych operacji (`put(...)`, `hashFunc(...)`, `doubleResize(...)`) w zależności od: metody haszowania, źródła danych. Rozważyć należy:

- metody: haszowania liniowego, dwukrotnego haszowania;
- dane: losowe łańcuchy znaków, dane z pliku (źródło MIT).

Lp.	Rozmiar	Czas[ms]	Końcowy wsp. α	put(...)	hashFunc(...)	doubleResize(...)
1.	5101	60,2	0,61262	218 599	477 633	5
2.	8297	59,7	0,37664	292 906	610 041	5
3.	16493	147	0,37895	285 547	598 209	4
4.	32909	123	0,37984	272 773	562 301	3
5.	65651	96,1	0,38080	247 716	496 882	2
6.	131111	125	0,38136	198 334	378 173	1
7.	263023	48	0,38019	100 000	130 738	0
8.	524309	23,5	0,19073	100 000	111 768	0

Tabela 1: Czas (i liczba) wykonywania operacji podczas wkładania **losowych danych** do tablicy mieszającej z **adresowaniem liniowym**.

Lp.	Rozmiar	Czas[ms]	Końcowy wsp. α	put(...)	hashFunc(...)	doubleResize(...)
1.	5101	267	0,61262	218 599	14 039 563	5
2.	8297	314	0,37664	292 906	23 239 711	5
3.	16493	279	0,37895	285 547	23 911 156	4
4.	32909	282	0,37984	272 773	18 758 229	3
5.	65651	203	0,38080	247 716	22 152 720	2
6.	131111	251	0,38136	198 334	20 693 853	1
7.	263023	33,6	0,38019	100 000	259 722	0
8.	524309	23,7	0,19073	100 000	131 104	0

Tabela 2: Czas (i liczba) wykonywania operacji podczas wkładania **danych z pliku** (MIT) do tablicy mieszającej z **adresowaniem liniowym**.

Komentarz do otrzymanych wyników (z trzech tabel): Dane losowe potrzebują mniejszą ilość czasu oraz wykonywania hashFunc, natomiast dla danych MIT potrzebna jest większa ilość czasu i wywołań hashFunc, pewnie dlatego że te dane są troszkę bardziej złośliwe i występuje większa ilość kolizji. Hashowanie dwukrotne dla danych MIT trochę mityguje złośliwość tych danych i spada też czas i ilość wywołania hashFunc do podobnych do danych losowych. Współczynnik Alfa się nie zmienia, a ilość wywołania funkcji put się mało zmienia.

Lp.	Rozmiar	Czas[ms]	Końcowy wsp. α	put(...)	hashFunc(...)	doubleResize(...)
1.	5101	100	0,61262	214 457	369 832	5
2.	8297	170	0,37664	280 390	470 451	5
3.	16493	184	0,37895	282 307	521 107	4
4.	32909	162	0,37984	272 773	446 314	3
5.	65651	105	0,38080	230 421	474 764	2
6.	131111	99,9	0,38136	198 334	306 727	1
7.	263023	36	0,38019	100 000	125 431	0
8.	524309	16,2	0,19073	100 000	110 669	0

Tabela 3: Czas (i liczba) wykonywania operacji podczas wkładania **danych z pliku** (MIT) do tablicy mieszającej z **dwukrotnym haszowaniem**.

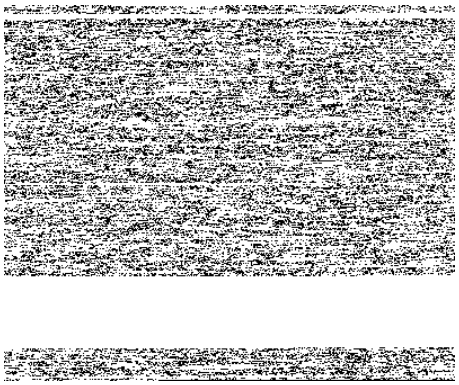
2.3 Zadanie 2

W pakiecie testowym `pl.edu.pw.ee.aisd2025zex3.performance.clustering` są cztery konkretne klasy do uruchomienia (jest sześć, ale nas interesują cztery). Wynikiem ich uruchomienia jest plik `PNG`, w którym czarne piksele są odzwierciedleniem wolnych komórek w tablicy mieszającej, a białe piksele – komórek zajętych.

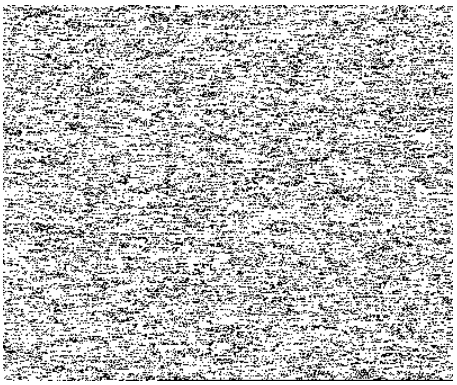
Plik jest zapisywany wierszami (czyli gdybyśmy pocięli ten plik na paski o szerokości jednego piksela i połączyli je, to otrzymalibyśmy „wizualizację" tablicy mieszającej).

Proszę utworzyć cztery obrazki wynikowe (zapisują się w korzeniu projektu) i zinterpretować je (proszę uzupełnić miejsce na komentarz).

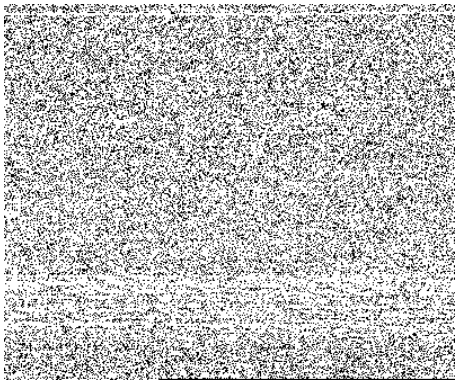
Komentarz do otrzymanych wizualizacji: TODO ——————.



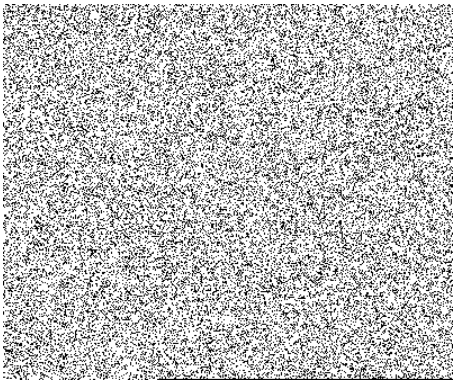
(a) Adresowanie liniowe, dane MIT



(b) Adresowanie liniowe, dane „losowe”



(c) Haszowanie dwukrotne, dane MIT



(d) Haszowanie dwukrotne, dane „losowe”

Rysunek 3: „Wizualizacja” wypełnienia hasza w zależności od typu danych i funkcji haszującej. (Białe piksele oznaczają zapełnione komórki, a czarne – puste.)

2.4 Zadanie 3

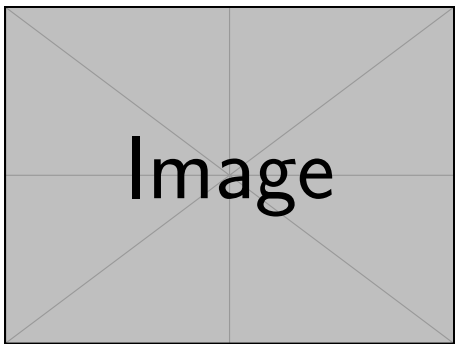
Wykorzystując pakiet testowy `pl.edu.pw.ee.aisd2026zex3.performance.clustering` proszę przygotować wizualizacje zapełniania tablic mieszających dla adresowania kwadratowego.

Proszę ręcznie dobierać (i zapisać!) stałe a oraz b w celu znalezienia najkorzystniejszego wyniku. Potrzebna będzie cierpliwość, intuicja i odrobina szczęścia.

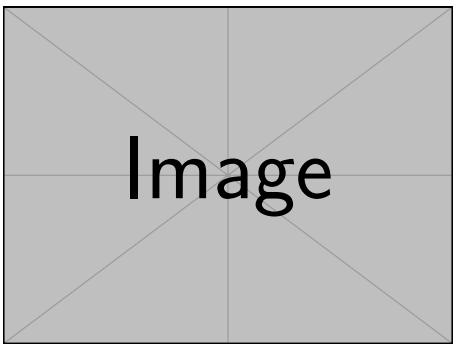
Proszę skupić się głównie na danych z MIT
(klasa: `HashQuadraticProbingMitWordsClusteringVisualization`). Dla znalezionej najlepszej pary stałych (a, b), proszę dodatkowo wykonać jedną wizualizację dla danych losowych
(klasa: `HashQuadraticProbingRandWordsClusteringVisualization`).

Proszę uzupełnić miejsce na komentarz (wnioski).

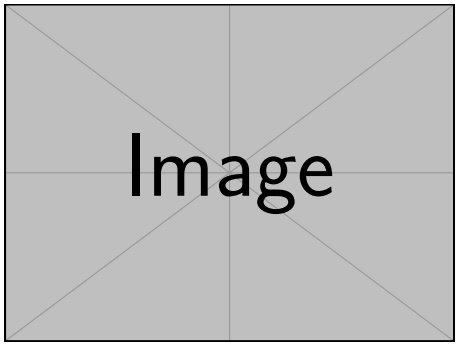
Komentarz do otrzymanych wizualizacji: TODO ——————.



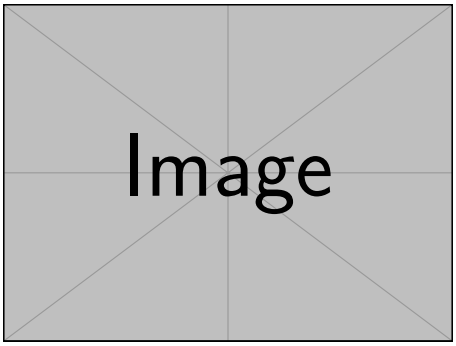
(a) Set_1. Dane **MIT**. a=??? | b=???



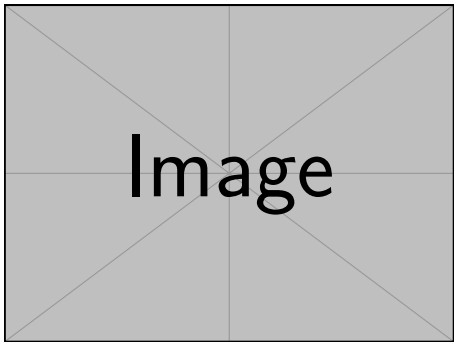
(b) Set_2. Dane **MIT**. a=??? | b=???



(c) Set_3. Dane **MIT**. a=??? | b=???



(d) Set_4. Dane **MIT**. a=??? | b=???



(e) Set_???. Dane **losowe**. a=??? | b=???

Rysunek 4: „Wizualizacja" wypełnienia hasza przy **adresowaniu kwadratowym**. Białe piksele oznaczają zapełnione komórki, a czarne – puste. Rysunki (a-d) pokazują wypełnienie dla danych MIT. Rysunek (e) pokazuje wynik wypełnienia danymi losowymi, dla wartości a oraz b wybranych jako najlepsze z pozostałych rysunków (a-d).

2.5 Zadanie 4

Proszę uzupełnić i przetestować implementację pod kątem wkładania, pobierania i usuwania elementów z tablicy mieszającej.

3 Na zakończenie

To już prawie koniec, ale (dzisiaj, podobnie jak ostatnio) nie wolno nam zapomnieć, że:

- wszystkie zmiany (cały kod źródłowy) muszą trafić do repozytorium;
- ostatnioą zmianę (*commit*), przed końcem zajęć, należy oznaczyć tagiem ze znacznikiem czasowym **CW_3_EOT**. A ostatnią zmianę (*commit*) należy oznaczyć tagiem ze znacznikiem czasowym **CW_3_FINAL**;
- końcową wersja tego dokumentu (PDF) należy umiecić w ISOD-zie.